THREE SIGMA

Unchained Gaming

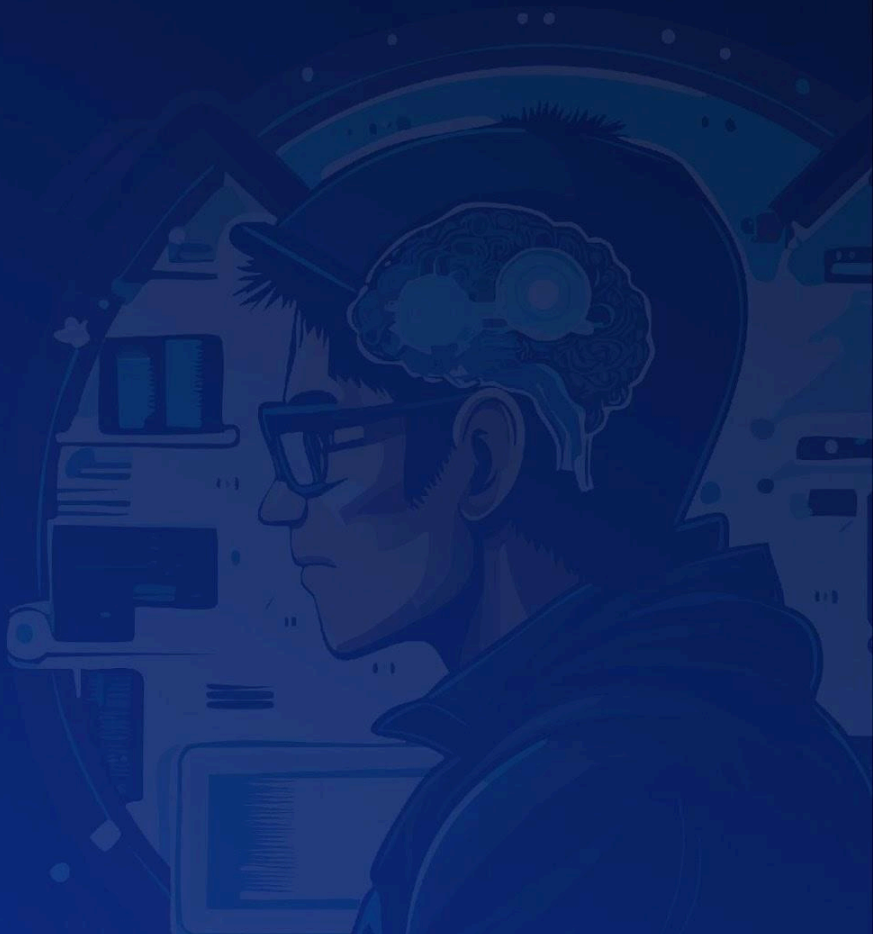
Security Review

Unchained

Disclaimer

Security Review

Unchained Gaming



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Unchained Gaming

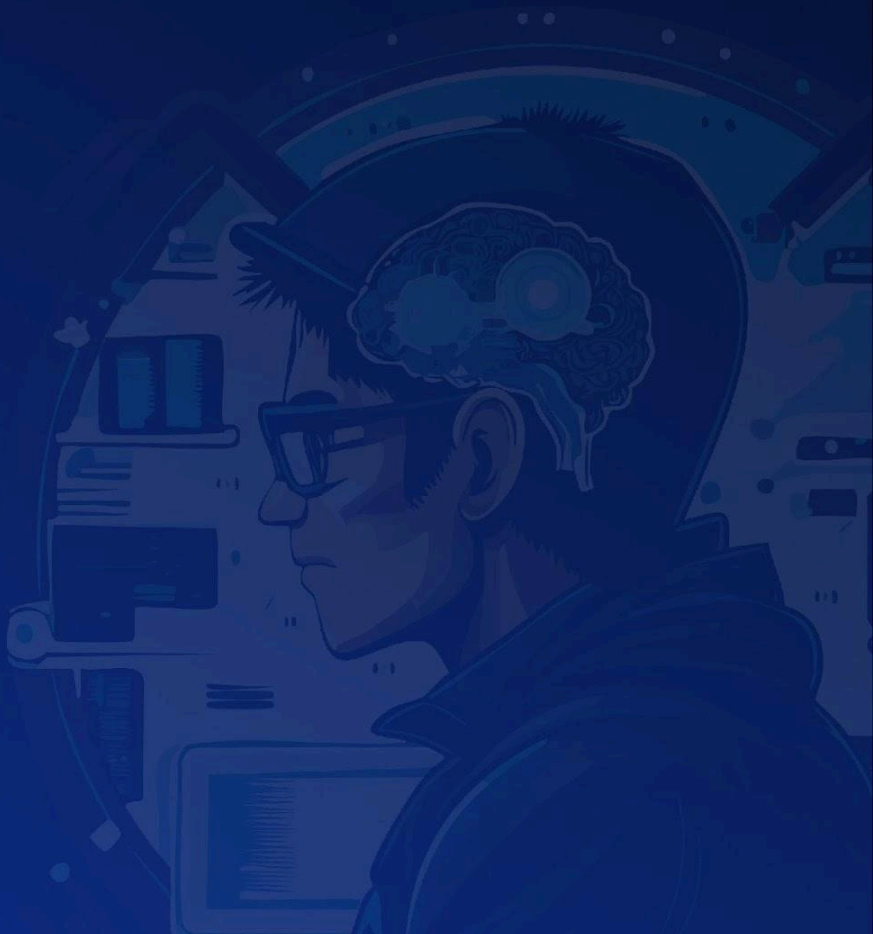
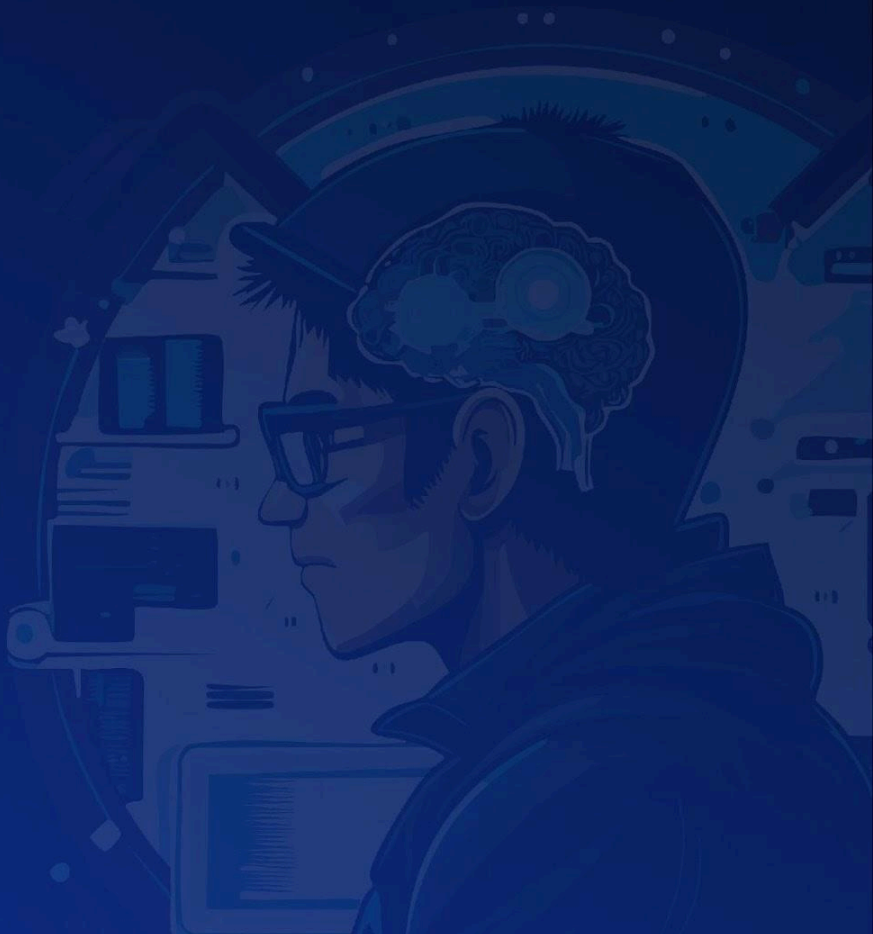


Table of Contents

Summary

Security Review

Unchained Gaming



Summary

Three Sigma audited Unchained in a 0.4 person week engagement. The audit was conducted on 08/02/2026.

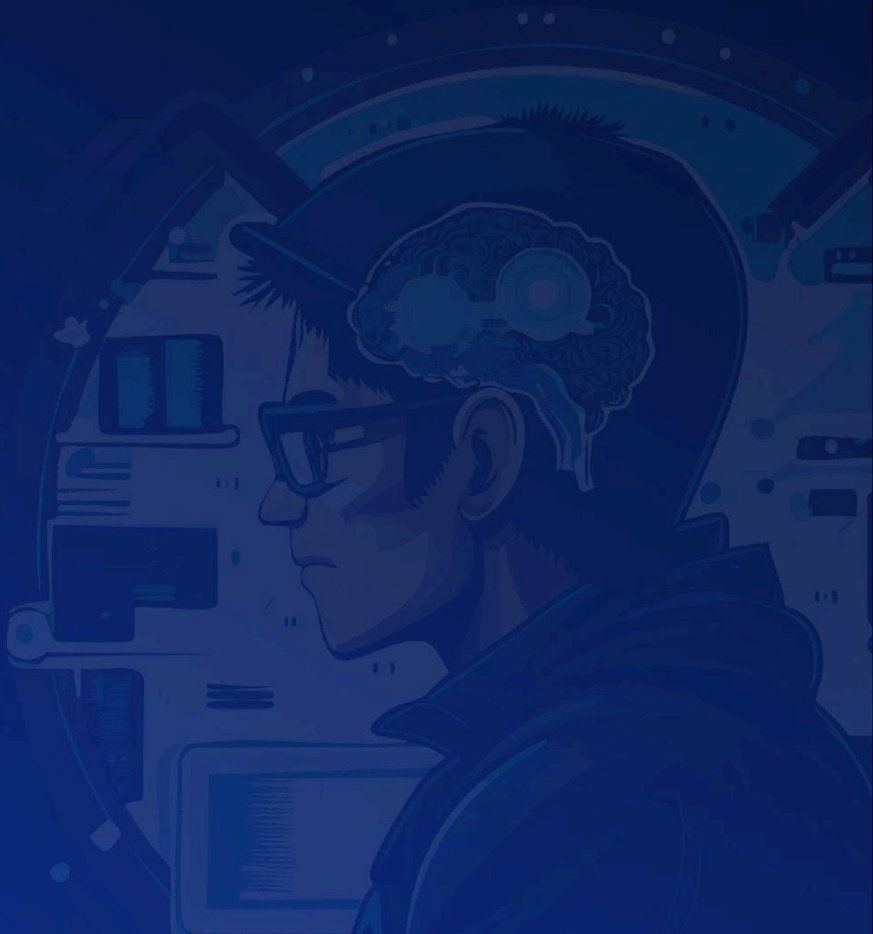
Protocol Description

Unchained is a game of wit and skill where players pay an entry fee to enter a Battle Royale against other players. The fees are pooled and redistributed to top-performing participants upon session completion. Players may fund their participation either through direct payment, previously accumulated winnings held within the contract, or via backend-sponsored entries ("Golden Tickets"), while a referral system incentivizes user acquisition by granting a share of entry fees to referring parties. The game also incorporates in-game collectible and equipment assets that can be purchased and traded between participants.

Scope

Security Review

Unchained Gaming



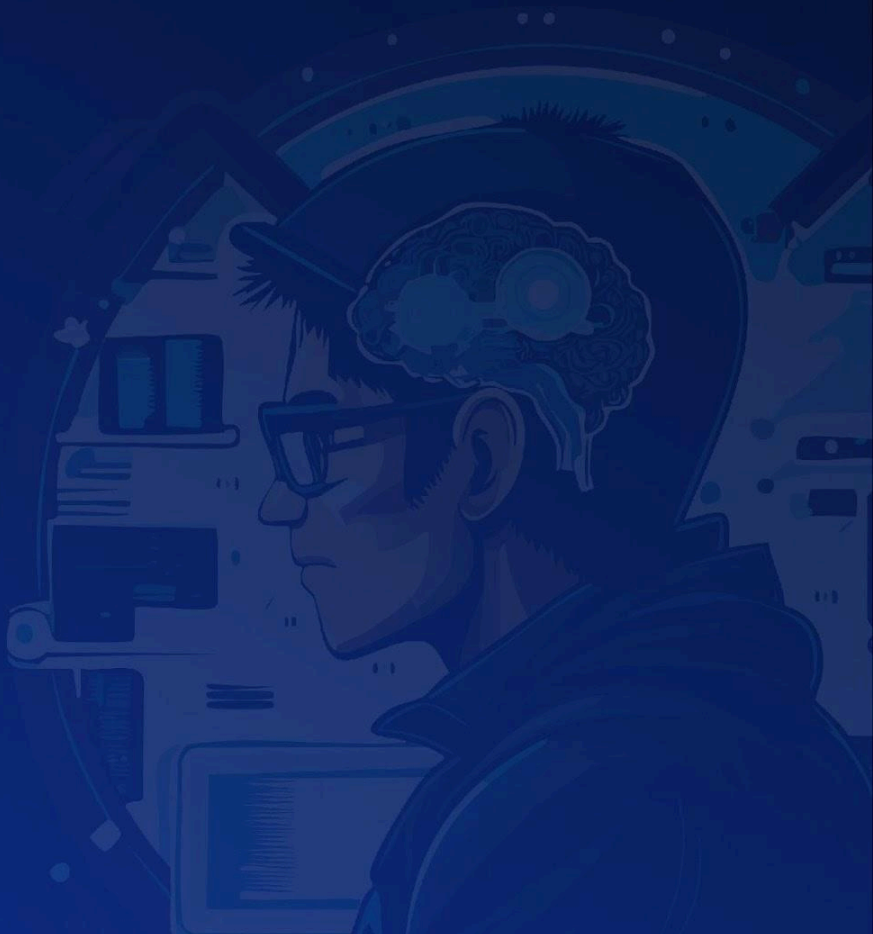
Scope

Filepath	nSLOC
src/SessionManager.sol	319
src/UnchainedCollectibles.sol	94
src/SessionManagerStorage.sol	38
Total	**451**

Methodology

Security Review

Unchained Gaming



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

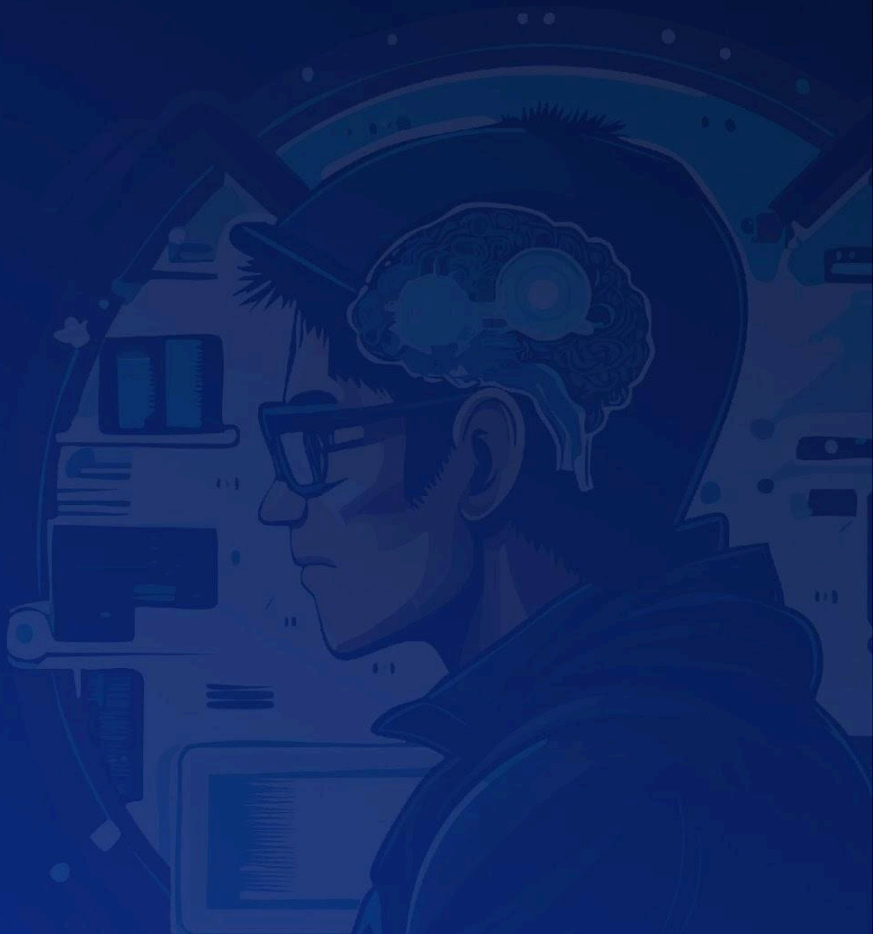
In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Unchained Gaming



Project Dashboard

Application Summary

Name	Unchained
Repository	https://github.com/bizarre-games/unchained-contracts
Commit	1808cd8
Language	Solidity
Platform	EVM

Engagement Summary

Timeline	08/02/2026
№ of Auditors	2
Review Time	0.4 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	1	1	0
High	2	2	0
Medium	1	1	0
Low	4	4	0
None	5	4	1

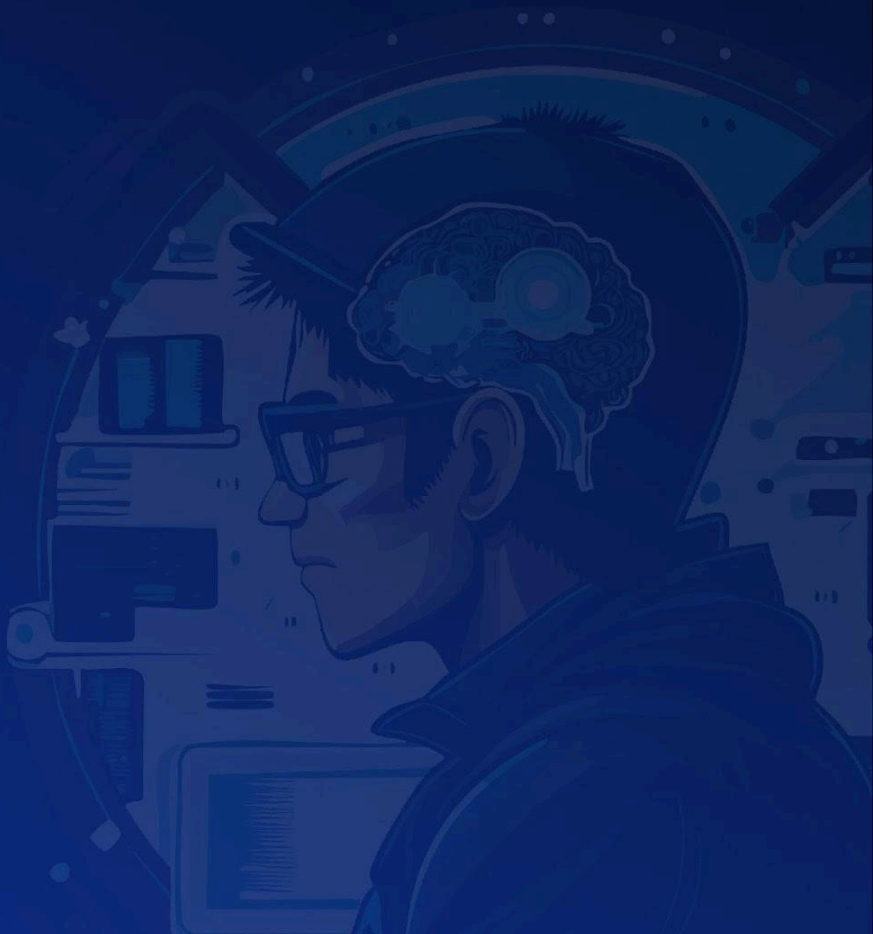
Category Breakdown

Suggestion	5
Documentation	0
Bug	8
Optimization	0
Good Code Practices	0

Risk Section

Security Review

Unchained Gaming



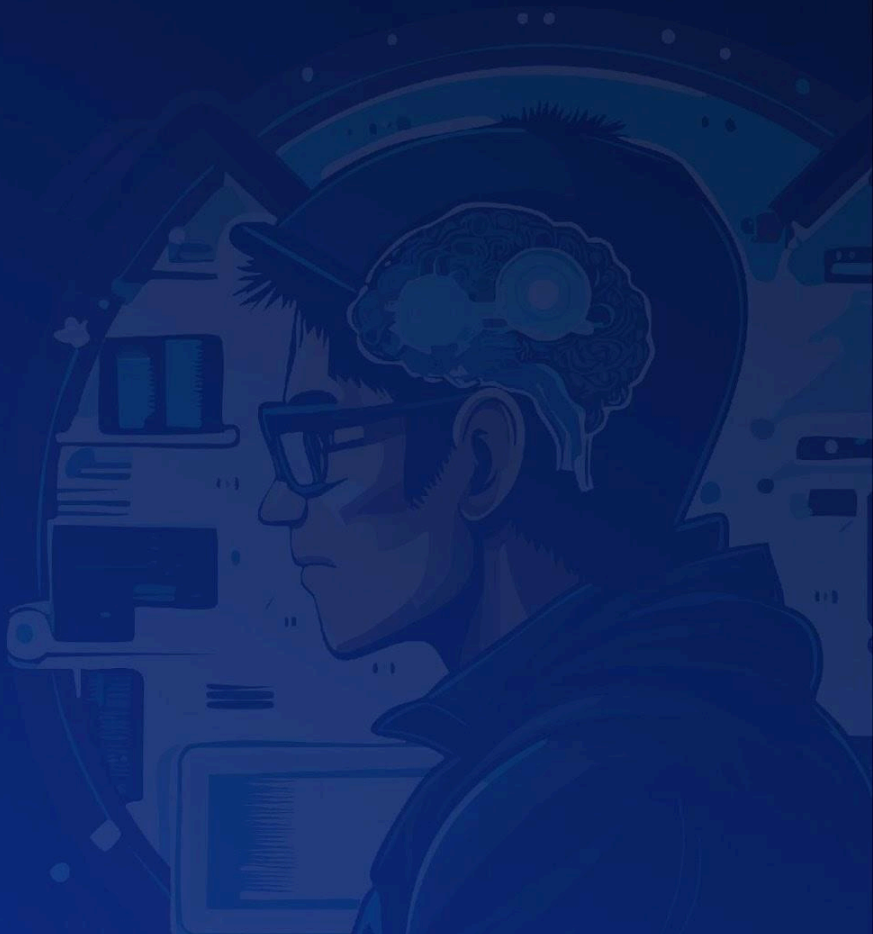
Risk Section

- **Off-chain winners selection**: The winner selection process is implemented off-chain and submitted to the smart contracts by a privileged role, meaning the protocol does not provide on-chain verifiability or cryptographic guarantees of fairness in the determination of session outcomes.

Findings

Security Review

Unchained Gaming



Findings

3S-Gaming-C01

Incorrect comparison in **claimRewards** allows referrers to double-claim session rewards

Id	3S-Gaming-C01
Classification	Critical
Impact	Critical
Likelihood	High
Category	Bug
Status	Addressed in #86d880a.

Description

The **SessionManager::claimRewards** function allows referrers to claim their accumulated referral rewards across a batch of finished sessions. To prevent double-claiming, the function tracks the highest session ID processed via **referrerLastSessionClaimed[referrer]** and requires that all provided session IDs exceed this checkpoint.

However, the function contains an incorrect comparison when computing the value to store as the new checkpoint. The loop uses **sessionIds[i] < lastSessionId**, which finds the **minimum** session ID in the provided array rather than the **maximum**. As a result, **referrerLastSessionClaimed[referrer]** is set to the smallest session ID in the batch instead of the largest, leaving all higher session IDs eligible to be claimed again in subsequent calls.

This allows a referrer to repeatedly drain rewards from the contract by re-claiming the same sessions.

Steps to reproduce

1. A referrer accumulates rewards across sessions 2, 3, and 4. The referrer's **referrerLastSessionClaimed** is currently **1**.

2. The referrer calls **SessionManager::claimRewards** with **sessionIds = [2, 3, 4]**.

3. Inside the loop, **lastSessionId** is initialized to **sessionIds[0]** which is **2**. The **<** comparison never finds a value smaller than **2**, so **lastSessionId** remains **2**.

4. At the end of the function, **referrerLastSessionClaimed[referrer]** is set to **2**.

5. The referrer calls **SessionManager::claimRewards** again with **sessionIds = [3, 4]**. Since both IDs are greater than **2**, the **require** check passes and rewards for sessions 3 and 4 are credited a second time.

6. The referrer can repeat step 5 with **sessionIds = [4]**, claiming session 4 a third time, and so on.

Recommendation

Change the comparison operator from **<** to **>** so that **lastSessionId** tracks the maximum session ID in the batch, ensuring all claimed sessions are properly marked as processed.

```
 function claimRewards(uint256[] memory sessionIds, address referrer) external {
    Storage storage $ = _loadStorage();
    uint256 lastSessionClaimed = $.referrerLastSessionClaimed[referrer];
    uint256 lastSessionId = sessionIds[0];
    uint256 rewards;
    for (uint256 i = 0; i < sessionIds.length; i++) {
-       if (sessionIds[i] < lastSessionId) {
+       if (sessionIds[i] > lastSessionId) {
            lastSessionId = sessionIds[i];
        }
        Session storage session = $.sessions[sessionIds[i]];
        require(lastSessionClaimed < sessionIds[i],
SessionAlreadyClaimed(lastSessionClaimed, sessionIds[i]));
        require(session.startTime > 0, SessionDoesNotExist(sessionIds[i]));
        require(session.endTime > 0, SessionNotFinished(sessionIds[i],
session.endTime, block.timestamp));
        require(
            session.endTime + CLAIM_DEADLINE <= block.timestamp,
            ClaimDeadlineNotReached(sessionIds[i], session.endTime +
CLAIM_DEADLINE, block.timestamp)
        );
        require(session.isCancelled == false, SessionCancelled(sessionIds[i]));
        rewards += session.referrerRewards[referrer];
    }
    $.referrerLastSessionClaimed[referrer] = lastSessionId;
    $.playerBalances[referrer] += rewards;
 }
```

3S-Gaming-H01

Unrestricted **claimRewards** allows anyone to permanently lock a referrer's unclaimed rewards

Id	3S-Gaming-H01
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #254f2a0.

Description

The **SessionManager::claimRewards** function is responsible for allowing referrers to collect their accumulated referral rewards across finished sessions. It accepts an array of **sessionIds** and a **referrer** address, aggregates the referrer's rewards from those sessions, and credits the total to the referrer's internal balance.

The function enforces a sequential claiming model through **referrerLastSessionClaimed[referrer]**, a cursor that tracks the minimum session ID from the most recent claim. Any future claim requires all provided **sessionIds** to be strictly greater than this cursor. However, **claimRewards** has no access control, meaning any external account can invoke it on behalf of any **referrer**.

This combination creates an attack vector where a malicious actor can selectively claim a subset of a referrer's sessions, advancing the cursor past unclaimed sessions and permanently making them unclaimable.

Steps to Reproduce:

1. Alice is a referrer with accumulated rewards in sessions 1, 3, and 4. Her **referrerLastSessionClaimed** is currently 0.

2. Bob (or any external account) calls **SessionManager::claimRewards** with **sessionIds = [4]** and **referrer = Alice**.

3. The function processes session 4, crediting only that session's rewards to Alice's balance, and sets **referrerLastSessionClaimed[Alice] = 4**.

4. Alice attempts to call **claimRewards** with **sessionIds = [1, 3]** to collect her remaining rewards.

5. The transaction reverts with **SessionAlreadyClaimed** because the require check **lastSessionClaimed < sessionIds[i]** evaluates **4 < 1** as false.

6. Alice's rewards for sessions 1 and 3 are permanently locked and unrecoverable.

Recommendation

Remove the sequential ordering check on **lastSessionClaimed** and instead prevent double-claiming by zeroing out **session.referrerRewards[referrer]** after each session's rewards are accounted for. The **referrerLastSessionClaimed** mapping is retained solely for off-chain tracking purposes via **SessionManager::getReferrerLastSessionClaimed**, and no longer serves as a security mechanism.

```
 function claimRewards(uint256[] memory sessionIds, address referrer) external {
     Storage storage $ = _loadStorage();
-    uint256 lastSessionClaimed = $.referrerLastSessionClaimed[referrer];
     uint256 lastSessionId = sessionIds[0];
     uint256 rewards;
     for (uint256 i = 0; i < sessionIds.length; i++) {
         if (sessionIds[i] < lastSessionId) {
             lastSessionId = sessionIds[i];
         }
         Session storage session = $.sessions[sessionIds[i]];
-        require(lastSessionClaimed < sessionIds[i],
SessionAlreadyClaimed(lastSessionClaimed, sessionIds[i]));
         require(session.startTime > 0, SessionDoesNotExist(sessionIds[i]));
         require(session.endTime > 0, SessionNotFinished(sessionIds[i],
session.endTime, block.timestamp));
         require(
             session.endTime + CLAIM_DEADLINE <= block.timestamp,
             ClaimDeadlineNotReached(sessionIds[i], session.endTime +
CLAIM_DEADLINE, block.timestamp)
         );
         require(session.isCancelled == false, SessionCancelled(sessionIds[i]));
         rewards += session.referrerRewards[referrer];
+         session.referrerRewards[referrer] = 0;
     }
     $.referrerLastSessionClaimed[referrer] = lastSessionId;
     $.playerBalances[referrer] += rewards;
 }
```

3S-Gaming-H02

Missing reentrancy protection in **mint** allows bypassing the item supply cap

Id	3S-Gaming-H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #c862386.

Description

UnchainedCollectibles::mint lets any user pay to mint a given amount of an ERC-1155 item. The function loads the item's **ItemData** into memory at line 56, validates that **data.minted + amount <= data.totalSupply** against this memory snapshot, and then calls **_mint(to, id, amount, "")** at line 61. The storage write **dataById[id].minted += amount** happens only after **_mint** returns at line 62. OpenZeppelin's ERC-1155 **_mint** internally invokes **ERC1155Utils.checkOnERC1155Received** on the **to** address, which transfers execution to the recipient's **onERC1155Received** hook. If **to** is a contract, it can re-enter **mint** during this callback. Because **dataById[id].minted** has not yet been updated, the supply check passes again using the stale **minted** value, and the attacker can mint beyond **totalSupply**. This can be repeated in a recursive loop until the attacker has minted an arbitrary number of tokens, limited only by gas. The same pattern exists in **UnchainedCollectibles::mintByOwner**, though exploitation there requires a compromised or malicious owner.

Recommendation

Move the **minted** storage update before the **_mint** call so that re-entrant invocations read the already-incremented counter:

```
  function mint(uint256 id, address to, uint256 amount) external payable
whenNotPaused {
      require(id < nextItemId, DoesNotExist(id));
      ItemData memory data = dataById[id];
      require(msg.value == data.price * amount, InsufficientPayment(id, data.price,
amount));
```

```
      if (data.totalSupply > 0) {
          require(data.minted + amount <= data.totalSupply, SupplyExceeded(id,
data.totalSupply, amount));
      }
+     dataById[id].minted += amount;
      _mint(to, id, amount, "");
-     dataById[id].minted += amount;
  }
```

The same fix should be applied to **UnchainedCollectibles::mintByOwner**.

3S-Gaming-M01

Reward distribution BIPS are not snapshotted at session creation

Id	3S-Gaming-M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #9d894e7.

Description

SessionManager::createSessions initializes new game sessions by setting the **entryFee**, **startTime**, and **maxPlayers** fields on the **Session** struct. It does not, however, snapshot the reward distribution parameters (**firstPlaceBIPS**, **secondPlaceBIPS**, **thirdPlaceBIPS**, **referrerBIPS**, **devBIPS**) into the session. These values are stored globally in the **Storage** struct and can be modified at any time by a game admin through **SessionManager::setRewards**.

When **SessionManager::finishSession** is called, it reads the BIPS values from the global **Storage** rather than from the individual session. If **SessionManager::setRewards** is invoked between session creation and session finalization, the reward distribution applied to the session will differ from what was in effect when players joined.

This also creates an inconsistency with referrer rewards. During joins, **referrerBIPS** is read from global **Storage** and used to compute per-referrer rewards stored in **session.referrerRewards**. When **SessionManager::finishSession** recalculates **totalReferrerReward**, it reads **referrerBIPS** from **Storage** again. If **referrerBIPS** changed between those two points, the **totalReferrerReward** deducted from the session balance will not match the sum of individual referrer rewards that were accumulated during joins, potentially causing an arithmetic underflow or leaving funds locked in the contract.

Recommendation

Snapshot the BIPS values into the **Session** struct at creation time and use the session-local values for all reward calculations.

Add BIPS fields to the **Session** struct in **SessionManagerStorage**:

```
 struct Session {
     uint256 balance;
     uint256 entryFee;
     uint256 startTime;
     uint256 endTime;
     address first;
     address second;
     address third;
     uint16 maxPlayers;
     uint16 playerCount;
     uint16 totalReferrerCount;
     bool isCancelled;
+    uint256 firstPlaceBIPS;
+    uint256 secondPlaceBIPS;
+    uint256 thirdPlaceBIPS;
+    uint256 referrerBIPS;
     mapping(address referrer => uint256 rewards) referrerRewards;
     mapping(address player => bool isJoined) playerJoined;
 }
```

Snapshot the values in **SessionManager::createSessions**:

```
 session.entryFee = data[i].entryFee;
 session.startTime = startTime;
 session.maxPlayers = data[i].maxPlayers;
+session.firstPlaceBIPS = $.firstPlaceBIPS;
+session.secondPlaceBIPS = $.secondPlaceBIPS;
+session.thirdPlaceBIPS = $.thirdPlaceBIPS;
+session.referrerBIPS = $.referrerBIPS;
```

Use session-local BIPS in **SessionManager::finishSession**:

```
-uint256 totalReferrerReward =
-    (session.entryFee * session.totalReferrerCount * $.referrerBIPS) /
BIPS_DENOMINATOR;
-uint256 firstPlaceReward = (session.balance * $.firstPlaceBIPS) /
BIPS_DENOMINATOR;
-uint256 secondPlaceReward = (session.balance * $.secondPlaceBIPS) /
BIPS_DENOMINATOR;
-uint256 thirdPlaceReward = (session.balance * $.thirdPlaceBIPS) /
BIPS_DENOMINATOR;
```

```
+uint256 totalReferrerReward =
+    (session.entryFee * session.totalReferrerCount * session.referrerBIPS) /
BIPS_DENOMINATOR;
+uint256 firstPlaceReward = (session.balance * session.firstPlaceBIPS) /
BIPS_DENOMINATOR;
+uint256 secondPlaceReward = (session.balance * session.secondPlaceBIPS) /
BIPS_DENOMINATOR;
+uint256 thirdPlaceReward = (session.balance * session.thirdPlaceBIPS) /
BIPS_DENOMINATOR;
```

Use the session's snapshotted **referrerBIPS** in all join functions (**SessionManager::joinSession**, **SessionManager::joinSessionFromContractBalance**, **SessionManager::joinSessionFromBackend**):

```
-_joinUpdates(session, sessionId, msg.sender, referrer, $.referrerBIPS);
+_joinUpdates(session, sessionId, msg.sender, referrer, session.referrerBIPS);
```

3S-Gaming-L01

Missing existence check in **cancelSession** allows pre-cancelling uninitialized session IDs

Id	3S-Gaming-L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #dd9fd9d.

Description

SessionManager::cancelSession verifies that **endTime == 0** and **isCancelled == false**, but does not check **session.startTime > 0**. Since all storage values default to zero for an uninitialized session, both guards pass for any session ID that has not yet been created. The function then writes **isCancelled = true** and **endTime = block.timestamp** to that slot. If **SessionManager::createSessions** is later called with the same session ID, it only checks **startTime == 0** (which still holds) so creation succeeds, but the session now inherits a non-zero **endTime** and **isCancelled = true**. The session enters a broken state: players can join (raised as issue **Missing cancellation check in _joinChecks allows players to join cancelled sessions**), but **finishSession** will always revert because **endTime != 0**. There is no **uncancelSession** function, so the deposited funds become recoverable only through admin-initiated **refundPlayers** calls.

Recommendation

Add an existence check to **SessionManager::cancelSession**:

```
  function cancelSession(uint256 sessionId) external onlyRole(GAME_ADMIN_ROLE)
{
      Session storage session = _loadStorage().sessions[sessionId];
+     require(session.startTime > 0, SessionDoesNotExist(sessionId));
      require(session.endTime == 0, SessionAlreadyFinished(sessionId,
session.endTime));
      require(session.isCancelled == false, SessionCancelled(sessionId));
      session.endTime = block.timestamp;
      session.isCancelled = true;
```

```
}
```

3S-Gaming-L02

Direct storage writes in **initialize** bypass validation enforced by setter functions

Id	3S-Gaming-L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #9c8ba8b.

Description

SessionManager::initialize sets the initial reward distribution (**devBIPS**, **referrerBIPS**, **firstPlaceBIPS**, **secondPlaceBIPS**, **thirdPlaceBIPS**) and the **treasury** address by writing directly to storage at lines 55–60. In contrast, **SessionManager::setRewards** enforces that the sum of all BIPS values equals **BIPS_DENOMINATOR** (10000), and **SessionManager::setTreasury** requires that the provided address is not **address(0)**. Since **initialize** does not call these functions and does not replicate their checks, a deployment with an incorrect BIPS split (e.g., values that do not sum to 10000) or a zero-address treasury would succeed silently. A misconfigured BIPS total would cause **SessionManager::finishSession** to either underflow when computing **devReward** or send an incorrect amount to the treasury. A zero-address treasury would result in ETH being irretrievably sent to **address(0)** on every **finishSession** call.

Recommendation

Change **setRewards** and **setTreasury** visibility from **external** to **public** and call them from **initialize** so that the same validation logic applies at deployment time:

```
- function setRewards(
+ function setRewards(
      uint256 firstPlaceBIPS,
      uint256 secondPlaceBIPS,
      uint256 thirdPlaceBIPS,
      uint256 referrerBIPS,
      uint256 devBIPS
- ) external onlyRole(GAME_ADMIN_ROLE) {
```

```
+ ) public onlyRole(GAME_ADMIN_ROLE) {


- function setTreasury(address _treasury) external onlyRole(ADMIN_ROLE) {
+ function setTreasury(address _treasury) public onlyRole(ADMIN_ROLE) {


  function initialize(address _treasury) public initializer {
      __AccessControl_init();
      _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
      _grantRole(ADMIN_ROLE, msg.sender);
      _grantRole(GAME_ADMIN_ROLE, msg.sender);
-     Storage storage $ = _loadStorage();
-     $.devBIPS = 700;
-     $.referrerBIPS = 300;
-     $.firstPlaceBIPS = 6000;
-     $.secondPlaceBIPS = 2000;
-     $.thirdPlaceBIPS = 1000;
-     $.treasury = _treasury;
+     setRewards(6000, 2000, 1000, 300, 700);
+     setTreasury(_treasury);
  }
```

3S-Gaming-L03

Missing cancellation check in **_joinChecks** allows players to join cancelled sessions

Id	3S-Gaming-L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #32081ec.

Description

SessionManager::_joinChecks is the shared validation gate for all three session join entry points (**joinSession**, **joinSessionFromContractBalance**, and **joinSessionFromBackend**). It verifies that the session exists, has not yet started, that the player has not already joined, and that the player cap has not been reached. However, it does not verify **session.isCancelled**. Since **SessionManager::cancelSession** sets **isCancelled = true** and writes **endTime = block.timestamp** but does not zero out **startTime**, a session that was cancelled before its start time still passes every check in **_joinChecks**; the **startTime** remains in the future and **endTime** is not evaluated. Players can therefore deposit entry fees into a session that has already been cancelled. The funds are not permanently lost because an admin can call **SessionManager::refundPlayers**, but the situation forces unnecessary admin intervention and creates a confusing user experience where a player pays into a session that will never run.

Recommendation

Add a cancellation check to **SessionManager::_joinChecks**:

```
  function _joinChecks(Session storage session, uint256 sessionId, address player)
internal view {
      require(session.startTime > 0, SessionDoesNotExist(sessionId));
+     require(!session.isCancelled, SessionWasCancelled(sessionId));
      require(session.startTime > block.timestamp, SessionStarted(sessionId));
      require(!session.playerJoined[player], PlayerAlreadyJoined(sessionId, player));
      require(session.maxPlayers > session.playerCount,
SessionMaxPlayersReached(sessionId, session.maxPlayers));
```

```
}
```

3S-Gaming-L04

Admin-gated refund for cancelled sessions leads to unnecessary trust dependency for fund recovery

Id	3S-Gaming-L04
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #4664876.

Description

SessionManager::refundPlayers is the only mechanism through which players can recover their entry fees after a session is cancelled via **SessionManager::cancelSession**. The function is restricted to the **GAME_ADMIN_ROLE** and accepts an array of player addresses to refund in batch. This means that players who deposited funds into a cancelled session have no ability to independently withdraw what they are owed — they must wait for an administrator to call **refundPlayers** on their behalf. This introduces a trust assumption that is unnecessary given that the cancellation state is already recorded on-chain via **session.isCancelled**. A player who can prove they joined a cancelled session (both conditions are verifiable from storage) should be able to claim their own refund without admin intervention. As currently implemented, if the admin key is lost, compromised, or simply unresponsive, players have no recourse to recover their deposited entry fees from cancelled sessions.

Recommendation

Add a permissionless function that allows a player to self-refund from a cancelled session by verifying **session.isCancelled** and **session.playerJoined[msg.sender]** directly. The existing admin-gated batch refund can be retained for convenience, but a self-service path should exist as a fallback.

```
+ function refundSelf(uint256 sessionId) external {
+     Storage storage $ = _loadStorage();
+     Session storage session = $.sessions[sessionId];
+     require(session.isCancelled == true, SessionCancelled(sessionId));
+     require(session.playerJoined[msg.sender], PlayerNotJoined(sessionId,
```

```
msg.sender));
+     session.playerJoined[msg.sender] = false;
+     session.balance -= session.entryFee;
+     session.playerCount--;
+     $.playerBalances[msg.sender] += session.entryFee;
+ }
```

3S-Gaming-N01

Missing uniqueness validation in **createItem** allows duplicate and colliding token URIs

Id	3S-Gaming-N01
Classification	None
Category	Suggestion
Status	Addressed in #33ffc93.

Description

UnchainedCollectibles::createItem is an owner-restricted function that registers a new collectible item by storing its metadata (**name**, **description**, **imageURI**, **rarity**, **tokenType**) and supply parameters into the **dataById** mapping at the next available ID. The function performs no validation on the provided string fields, meaning the owner can create multiple distinct token IDs that share identical metadata. Beyond exact duplicates, **UnchainedCollectibles::uri** constructs the token URI by concatenating all string fields through **abi.encodePacked**. Since **abi.encodePacked** does not length-prefix dynamic types, adjacent string fields can produce the same encoded output from different inputs — for example, a **name** of **"ab"** with a **description** of **"c"** produces the same packed encoding as a **name** of **"a"** with a **description** of **"bc"**. This means two items with different metadata can resolve to identical URIs, causing confusion for marketplaces, indexers, and users that rely on URI uniqueness to distinguish tokens.

Recommendation

Compute a hash of the resulting URI (or of the constituent fields) and track it in a mapping to reject duplicates at creation time. The hash should be derived from **abi.encode** rather than **abi.encodePacked** to avoid the same collision issue in the deduplication check itself:

```
+ mapping(bytes32 => bool) private uriUsed;


 function createItem(
     uint256 totalSupply,
     uint256 price,
     string calldata name,
     string calldata description,
```

```
      string calldata imageURI,
      string calldata rarity,
      string calldata tokenType
  ) external onlyOwner {
+     bytes32 uriHash = keccak256(abi.encode(name, description, imageURI, rarity,
tokenType));
+     require(!uriUsed[uriHash], "Duplicate item");
+     uriUsed[uriHash] = true;
      dataById[nextItemId] = ItemData({
```

3S-Gaming-N02

Unused **devBIPS** parameter misrepresents the actual developer reward percentage

Id	3S-Gaming-N02
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **SessionManager** contract manages sessions where players pay an entry fee to participate. When a session ends, **SessionManager::finishSession** distributes the accumulated session balance among the top three winners, referrers, and the protocol treasury. The developer (treasury) reward is computed as the residual balance remaining after subtracting winner rewards and referrer rewards from the total session balance.

The **SessionManager::setRewards** function accepts a **devBIPS** parameter and enforces that all BIPS values sum to **BIPS_DENOMINATOR** (10000).

However, **devBIPS** is never referenced in **SessionManager::finishSession**. The developer reward is instead derived from the leftover balance:

```
uint256 devReward =
    (session.balance - totalReferrerReward - firstPlaceReward -
secondPlaceReward - thirdPlaceReward);
```

Because referrer rewards are only allocated for players who have a referrer, unclaimed referrer portions are absorbed into the developer reward. This causes the developer's actual share to exceed the configured **devBIPS** value whenever any player lacks a referrer.

Steps to Reproduce

1. Configure rewards via **SessionManager::setRewards** with: **devBIPS** = 700 (7%), **referrerBIPS** = 300 (3%), **firstPlaceBIPS** = 6000 (60%), **secondPlaceBIPS** = 2000 (20%), **thirdPlaceBIPS** = 1000 (10%).

2. Create a session with an entry fee of 100.

3. Three players join: Alice (with a referrer), Bob (no referrer), and Charlie (no referrer). The session balance is 300.

4. A game admin calls **SessionManager::finishSession**. The rewards are calculated as:

   - **totalReferrerReward** = 100 * 1 * 300 / 10000 = 3 (only Alice had a referrer)

   - **firstPlaceReward** = 300 * 6000 / 10000 = 180

   - **secondPlaceReward** = 300 * 2000 / 10000 = 60

   - **thirdPlaceReward** = 300 * 1000 / 10000 = 30

   - **devReward** = 300 - 3 - 180 - 60 - 30 = **27**

5. The effective developer reward is 27 / 300 = **9%**, not the configured 7%. The **devBIPS** variable is stored but has no effect on the actual reward distribution.

Recommendation

Remove the **devBIPS** parameter from **SessionManager::setRewards** and update the validation to use **<=** instead of **==**, since the developer reward is inherently the residual balance and does not need an explicit BIPS configuration. Also remove the **devBIPS** storage variable from **SessionManagerStorage**.

```
    function setRewards(
        uint256 firstPlaceBIPS,
        uint256 secondPlaceBIPS,
        uint256 thirdPlaceBIPS,
-       uint256 referrerBIPS,
-       uint256 devBIPS
+        uint256 referrerBIPS
    ) external onlyRole(GAME_ADMIN_ROLE) {
        require(
-           firstPlaceBIPS + secondPlaceBIPS + thirdPlaceBIPS + referrerBIPS +
devBIPS == BIPS_DENOMINATOR,
-           InvalidRewards(firstPlaceBIPS + secondPlaceBIPS + thirdPlaceBIPS +
referrerBIPS + devBIPS)
+           firstPlaceBIPS + secondPlaceBIPS + thirdPlaceBIPS + referrerBIPS <=
BIPS_DENOMINATOR,
+           InvalidRewards(firstPlaceBIPS + secondPlaceBIPS + thirdPlaceBIPS +
referrerBIPS)
        );
        Storage storage $ = _loadStorage();
        $.firstPlaceBIPS = firstPlaceBIPS;
        $.secondPlaceBIPS = secondPlaceBIPS;
        $.thirdPlaceBIPS = thirdPlaceBIPS;
        $.referrerBIPS = referrerBIPS;
-       $.devBIPS = devBIPS;
```

```
}
```

3S-Gaming-N03

Semantically incorrect error in **refundPlayers** leads to misleading revert reason

Id	3S-Gaming-N03
Classification	None
Category	Suggestion
Status	Addressed in #63f5a56.

Description

SessionManager::refundPlayers is an admin-gated function that credits entry fees back to players after a session has been cancelled. At line 175, it guards execution with **require(session.isCancelled == true, SessionCancelled(sessionId))**. This reverts with **SessionCancelled** when the session is **not** cancelled, the exact opposite of what the error name communicates. A caller or off-chain integration receiving this revert would reasonably interpret **SessionCancelled** as confirmation that the session was cancelled, when in reality the revert signals that the session has not been cancelled and therefore cannot be refunded. This complicates debugging and error handling for any frontend or monitoring system parsing revert data.

Recommendation

Introduce a dedicated error with the correct semantics and use it in the require statement:

```
+ error SessionNotCancelled(uint256 sessionId);
```

```
- require(session.isCancelled == true, SessionCancelled(sessionId));
+ require(session.isCancelled == true, SessionNotCancelled(sessionId));
```

3S-Gaming-N04

Unwritten **playerReferrer** mapping leads to non-functional sticky referrer logic

Id	3S-Gaming-N04
Classification	None
Category	Suggestion
Status	Addressed in #96b581e.

Description

The **SessionManagerStorage** contract declares a **playerReferrer** mapping (storage slot at **SessionManagerStorage.sol:45**) intended to persist a player's referrer address across sessions. All three join entry points — **SessionManager::joinSession**, **SessionManager::joinSessionFromContractBalance**, and **SessionManager::joinSessionFromBackend** — read from **$.playerReferrer[msg.sender]** (or **$.playerReferrer[player]**) before falling back to the **_referrer** parameter passed by the caller. However, no function in the contract ever writes to **$.playerReferrer**. Because Solidity mappings return **address(0)** for uninitialized keys, the lookup will always resolve to the zero address, the **if (referrer == address(0))** branch will always be taken, and the caller-supplied **_referrer** will always be used. The mapping read and conditional check are therefore dead code, and what appears to be a "sticky referrer" feature — where a player's first referrer is remembered and reused in future sessions — is silently non-functional. A player can effectively switch referrers on every join call, which may not be the intended economic design.

Recommendation

If the feature is not needed, remove the **playerReferrer** mapping from **Storage** and the associated read logic to reduce contract size and avoid confusion.

If instead the sticky referrer feature is desired, persist the referrer on first assignment by writing to the mapping inside the join functions. In such case, the users should also be offered a function to replace the saved referrer.

3S-Gaming-N05

Redundant zero balance check in **SessionManager::withdrawBalanceForUser**

Id	3S-Gaming-N05
Classification	None
Category	Suggestion
Status	Addressed in #8be9131.

Description

The **SessionManager::withdrawBalanceForUser** function allows a **GAME_ADMIN_ROLE** holder to withdraw accumulated balances on behalf of a user. It calls the internal **SessionManager::_withdraw** helper to execute the balance transfer.

Before delegating to **_withdraw**, **withdrawBalanceForUser** explicitly checks that the user's balance is non-zero via **require($.playerBalances[user] != 0, ZeroBalance(user))**. However, **_withdraw** already performs the identical check (**require($.playerBalances[to] != 0, ZeroBalance(to))**), making the check in **withdrawBalanceForUser** redundant. This results in unnecessary gas consumption on every successful call due to the duplicated storage read and comparison.

Note that the other external entry point, **SessionManager::withdrawBalance**, does not include this redundant check and relies solely on the validation within **_withdraw**.

Recommendation

Remove the redundant zero-balance check from **SessionManager::withdrawBalanceForUser** and rely on the existing validation in **SessionManager::_withdraw**.

```
 function withdrawBalanceForUser(address user) external
onlyRole(GAME_ADMIN_ROLE) {
-    Storage storage $ = _loadStorage();
-    require($.playerBalances[user] != 0, ZeroBalance(user));
     _withdraw(user);
 }
```